

AAVAIL Revenue Forecasting

Part 2 — Model Comparison, Selection & Deployment Prep

Predicting total revenue over the next 30 days, at any point in time, for the company overall and for each of the ten highest-revenue countries.

Task	Supervised regression: predict revenue summed over days $t+1 \dots t+30$, as of any date t
Series modeled	11 (overall + 10 countries), one model each
Models compared	Naive, seasonal-naive, Ridge, Random Forest, HistGBM, LightGBM
Selected approach	hist_gbm — {'max_depth': 3, 'learning_rate': 0.1, 'max_iter': 100}

1. Modeling Approaches Compared

Part 1 established that next-month revenue is the target and that ten countries account for the large majority of it. Following the case-study guidance, every candidate here is standardized on the same task: given all data available as of a date t , predict total revenue over the following 30 days ($t+1 \dots t+30$) — a single number per as-of date, which turns the forecasting problem into ordinary supervised regression on engineered features rather than a multi-step sequence prediction problem.

- **Naive (persistence) baseline** — next 30 days = trailing 28 days of revenue. Stands in for the “same as last month” estimate management currently uses informally.
- **Seasonal-naive baseline** — next 30 days = the same 30-day window one year earlier, falling back to the naive baseline where a year of history isn't yet available.
- **Ridge regression** — a regularized linear model. Included deliberately despite the case-study's own caution that OLS assumptions break down on autocorrelated, revenue-derived features; regularization was expected to help, and it's a useful check on whether the non-linear models are earning their complexity.
- **Random Forest** — a bagged tree ensemble, robust to the feature scale differences across an 11-series, very-different-revenue-scale problem (UK vs. Singapore).
- **Histogram-based Gradient Boosting (scikit-learn)** and **LightGBM** — two gradient-boosted tree implementations, generally the strongest performers on tabular, engineered time-series features of this kind.

Feature engineering is shared across every learned model: for revenue, purchases, unique streams, unique customers, and total views, trailing 7/14/28/70-day sums and means, a 28-day trend, and a 30-day-value-one-year-ago feature (flagged when unavailable rather than dropping the row — most series don't have a full year of prior history). Calendar features (day-of-week, day-of-month, month, and an explicit end-of-year-gap flag) encode the seasonality Part 1 found in the data. See `aavail/features.py`.

1.1 Why purchases and views are in the feature set, not just lagged revenue

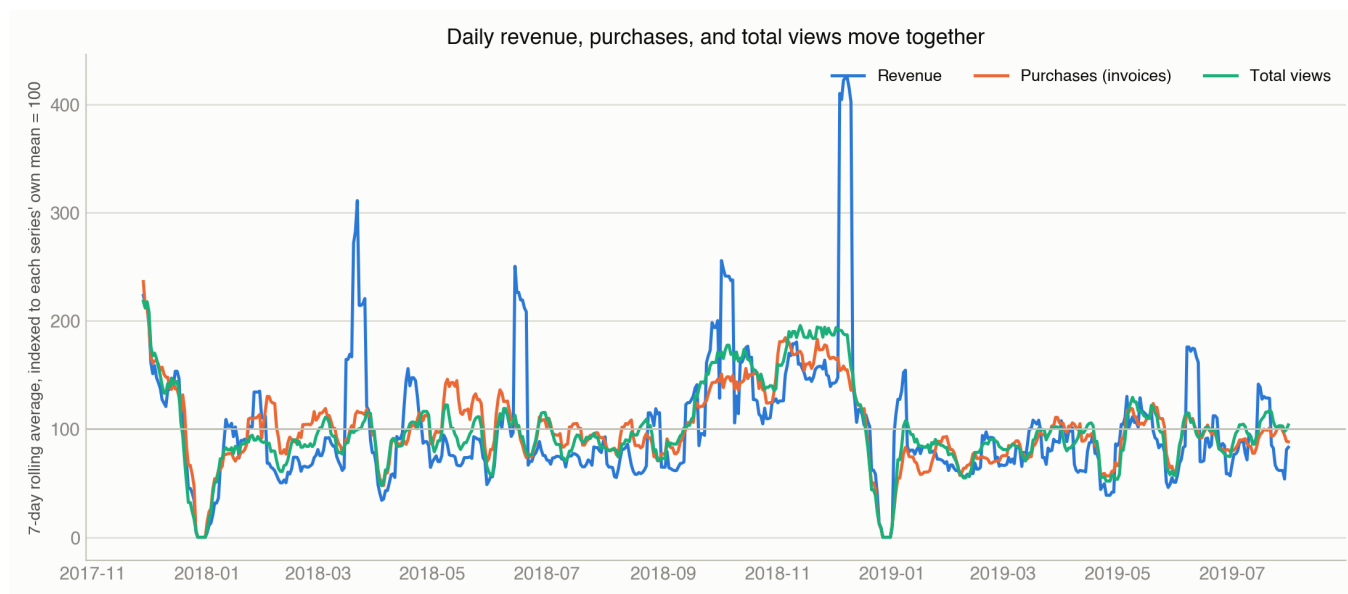


Figure 9. Daily revenue, purchases, and total views — 7-day rolling averages, each indexed to its own mean = 100 so all three share one axis despite very different raw scales.

Pivoted to a daily interval, the three series move together: purchases and total views are almost identical in shape (correlation 0.93), since more distinct purchases mechanically means more content streamed. Revenue tracks the same broad seasonal pattern — the January dip, the autumn build-up, the December gap — but correlates more loosely with the other two (0.62–0.70), because revenue alone carries occasional large single-invoice spikes (the tall blue peaks in Mar, Jun, and Oct/Nov 2018) that purchase counts and view counts don't share. That's the same heavy-tailed price effect Part 1 found in the transaction-price distribution, and it's the concrete reason purchases and views were engineered as separate features here rather than assumed redundant with lagged revenue.

2. Iteration

Hyperparameters were tuned once, on the overall series (it has the most history), using 4-fold `TimeSeriesSplit` cross-validation — folds respect chronological order so no model is ever validated on data that precedes its training window. A small, deliberately cheap grid was searched per model family (e.g. tree depth, learning rate, number of estimators/leaves for the boosted models; regularization strength for Ridge); the winning configuration per family was then reused unchanged across all 11 series, keeping the comparison apples-to-apples rather than re-tuning per country.

Each of the 11 series was then evaluated on its own chronological holdout (the most recent ~20% of usable rows, minimum 40), so every model is judged on data it never trained on.

3. Comparison Results

3.1 Which model family wins, on average

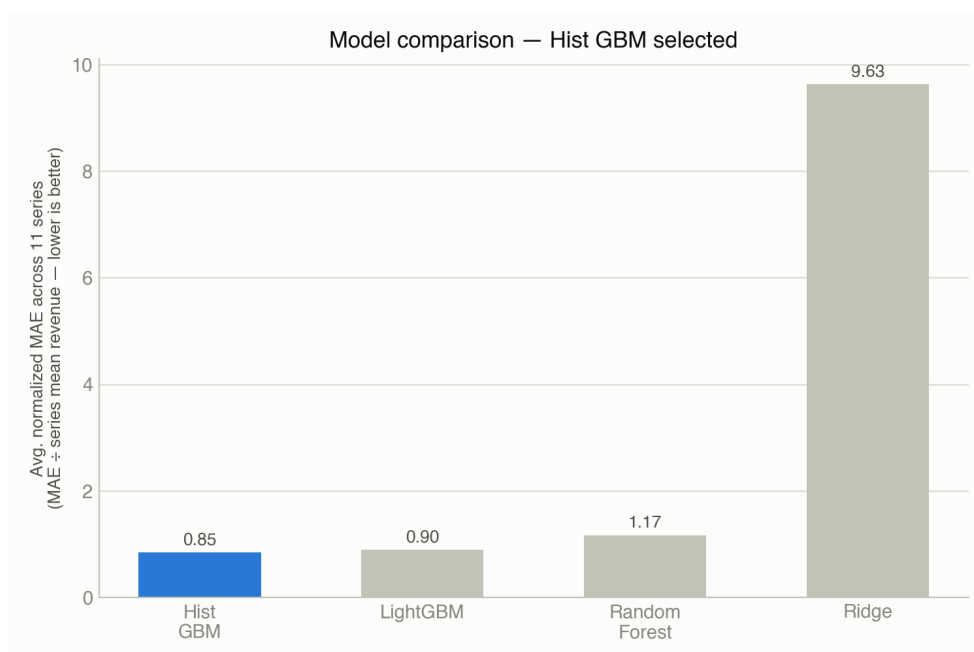


Figure 10. Mean absolute error, normalized by each series' average revenue and averaged across all 11 series (lower is better).

Histogram Gradient Boosting had the lowest average normalized error (0.85), narrowly ahead of LightGBM (0.90). Random Forest trailed (1.17), and Ridge was far worse on average (9.63) — almost entirely because it produced a wildly unstable extrapolation on Norway, a country whose revenue is concentrated in one or two outsized months (flagged as a lumpy market in Part 1). That instability is itself a finding: linear models extrapolate badly on the sparse, spiky countries, which is a concrete reason — not just an average-case one — to prefer a tree-based approach here.

3.2 Does the winner actually beat the naive baseline, everywhere?

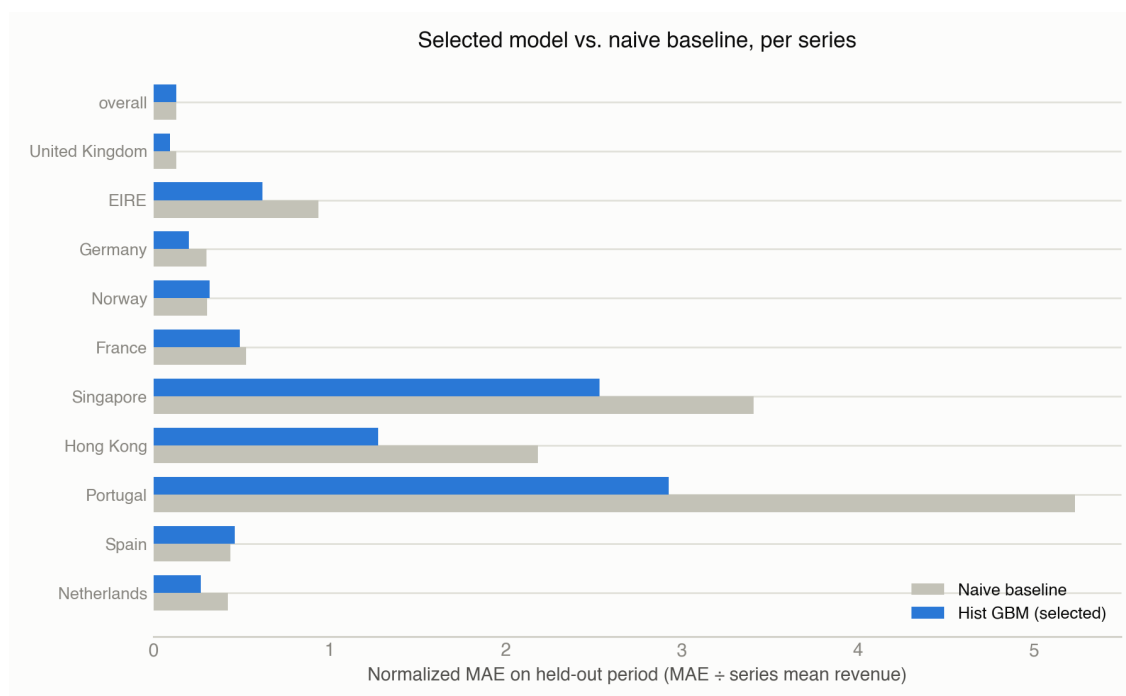


Figure 11. Normalized MAE, naive baseline vs. the selected model (Histogram Gradient Boosting), per series.

For the steady, high-volume series (overall, UK, Germany, Norway, France, Spain, Netherlands) the model matches or modestly beats the naive baseline. For the lumpiest, lowest-volume markets (EIRE, Singapore, Hong Kong, Portugal) both the model and the naive baseline show much higher relative error — the model still generally edges out naive, but neither is highly accurate, because a handful of outsized invoices make 30-day totals hard to predict from any historical pattern. This is the clearest evidence in this report that Part 1's per-country accuracy caveat (H5) was warranted: the service should report wider uncertainty, or a simpler baseline, for these markets rather than implying UK-level confidence.

4. Selected Approach, Retraining, and Deployment Readiness

Histogram-based Gradient Boosting (`{'max_depth': 3, 'learning_rate': 0.1, 'max_iter': 100}`) was selected: best average accuracy, no catastrophic failure modes on any series, and (unlike LightGBM here) no extra tuning needed to close the gap. Once selected, the same model family and hyperparameters were retrained on **100% of available history** for every one of the 11 series — the held-out evaluation above exists purely to choose and validate the approach; the deployed models use all the data, since forecasting accuracy is monotonic in training data for this kind of model.

4.1 Sanity check: overall revenue, actual vs. predicted

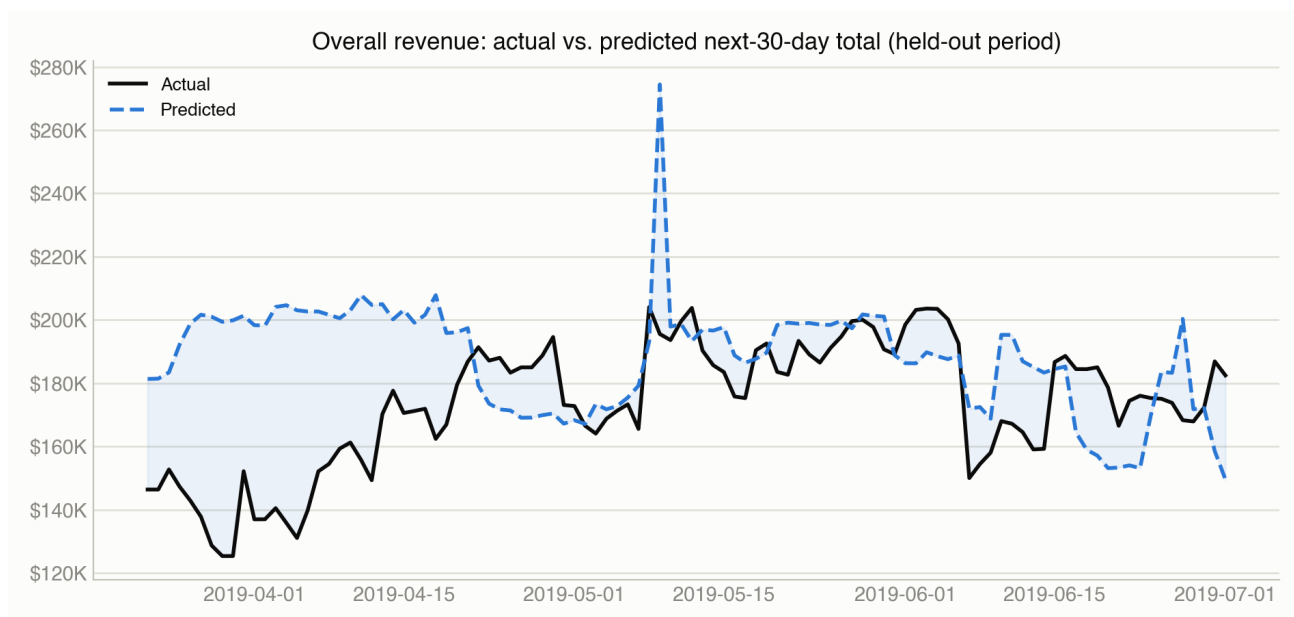


Figure 12. Held-out overall revenue: actual next-30-day total vs. the model's prediction.

On the overall series' holdout, the selected model's mean absolute error was \$25,452 against an average 30-day revenue of \$173,247 — roughly a 16% typical error. The predicted line tracks the level and direction of actual revenue well; it lags sharp single-week spikes (early May) since those are driven by a few large invoices no engineered feature fully anticipates.

4.2 What the model actually relies on

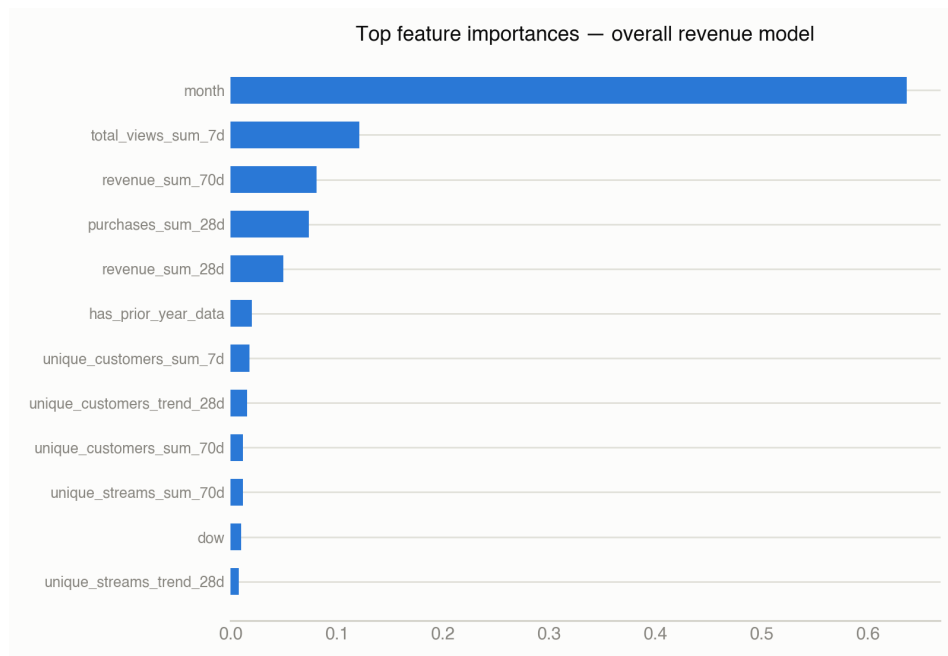


Figure 13. Permutation feature importance, overall revenue model (top 12 features).

Calendar month dominates — consistent with Part 1's finding of a strong autumn build-up toward the holiday season. Trailing views, trailing revenue, and trailing purchase counts follow, confirming Part 1's H3: recent activity carries real predictive signal beyond raw lagged revenue. This also means the model is depending on a broad mix of features rather than one brittle signal, which is reassuring for deployment.

Deployment artifacts

- 11 trained models (one per series) saved via joblib to `models/*.joblib`.
- `models/metadata.json` records the model family, hyperparameters, the exact feature-column order each model expects, and the training date range per series — everything an inference service needs without re-deriving it.
- `aavail/features.py` exposes `latest_feature_row()`, which builds the single feature row for 'today' from a time series, so training and inference share identical feature logic and can't silently drift apart.
- Verified end-to-end: loading a saved model, building today's feature row, and predicting next-30-day revenue runs correctly for both the overall series and an individual country (see the Part 2 training script's smoke test).

5. Summary and What's Next

- Six candidate approaches were compared — two baselines and four learned models — on an identical 30-day-ahead supervised task across 11 series.
- Histogram Gradient Boosting was the most accurate on average and the most robust: it did not blow up on the lumpy, low-volume countries the way Ridge did.
- The model beats a naive persistence baseline on the high-volume, steady series but only modestly improves on it for the sparsest markets — a limitation worth stating to management directly rather than masking behind an aggregate accuracy number.
- All 11 models are retrained on full history and saved with the metadata an inference service needs, ready for Part 3 (packaging this as the actual prediction service, with monitoring against the cs-production data held out from this project).

Reproduce via `scripts/train_models.py` (**comparison + retrain**) and `reports/build_model_figures.py` (**figures**); **source metrics live in** `reports/content/model_comparison.json`.